
Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 1: Metamodelling
Project Type: Code Generation
September 16, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1	Introduction and Scope	1
1.1	Response to the Task 0 Feedback	1
1.2	What Is In Scope and What Is Not	1
2	Domain Analysis	1
2.1	Where the Repetition Is	1
2.2	The Concepts the Domain Requires	2
2.3	Notation	2
3	Modelling Decisions	2
3.1	Three Model Stages, Two Metamodels	2
3.2	The Layer Rule	3
3.3	Contention Decides Authority	3
3.4	Derivation Is Total, and There Is No Override	3
3.5	Values Name Model Concepts, Never Target Fields	3
3.6	Closed Vocabularies	3
3.7	Structure Carries Atomicity; the Graph Carries Order	3
4	Alternatives Considered	4
4.1	A Third Metamodel for the Generated Files	4
4.2	Grammar First or Metamodel First	4
4.3	One Resolved Model, or One Per Application	4
5	The Source Metamodel	5
5.1	The Project Document	5
5.2	The Platform Document	5
5.3	Closed Vocabularies	5
6	The Target Metamodel	10
6.1	What It Must Record	10
6.2	Structure	10
6.3	The Path Plan	10
6.4	The Generated Resources	11
6.5	No Target Field Names	11
7	Example Models	11
7.1	The Minimal Model	11
7.2	Refused Models	16
7.3	A Resolved Model as a Deployment View	16
7.4	Example Target Models	16

1 Introduction and Scope

Task 0 selected the code-generation problem: generate a complete Kubernetes deployment set from one abstract model of what an application needs, rather than from hand-maintained YAML in which the same fact is written down several times. This report covers Task 1, the metamodeling step. It describes the domain, the modelling decisions and the alternatives that were considered, then presents the two metamodels and the example models that instantiate them.

1.1 Response to the Task 0 Feedback

The feedback on Task 0 raised three points, and each changed something in this report.

Scope. The warning was to be careful with scope, and it was justified: the original proposal carried three metamodels and an open-ended set of generated resources. The scope is now fixed by three statements. The pipeline has *two* metamodels, not three (Section 3): the Deliverable Set is generated text, so it is not given a metamodel of its own. The generated resources are a *closed* set of six generators, listed in Section 6, and adding a seventh is a recorded decision rather than a new feature. The estate modelled is a *single* cluster, so every rule that would only become visible at a second cluster is recorded as an assumption instead of being designed for.

Notation. The question was why UML class diagrams rather than a deployment or component diagram, and whether ArchiMate or the C4 model would serve. Section 2.3 answers both: a class diagram is the notation for a language’s abstract syntax and the others describe systems rather than languages, so they cannot replace it for the metamodels; but for the example *models* a deployment view is the better picture, and Section 7 now carries one.

1.2 What Is In Scope and What Is Not

In scope	Out of scope
Two metamodels: the authored source language, and the resolved target model.	A metamodel of the generated files. They are text, and are compared as text.
A closed set of six generators for one cluster.	Additional target platforms, and a second cluster.
Validation of a source model, and the constraints that refuse an invalid one.	Applying the generated files to a cluster, which is a separate concern.
Dependency resolution between applications.	Runtime behaviour, scaling decisions and monitoring rules.

Table 1: Scope of the Project After the Task 0 Feedback

The last exclusion is the one that most affects the report’s claims. This project generates a deployment set; it does not deploy it. Every statement about correctness in this report is therefore a statement about the generated model and the generated files, never about a running cluster.

2 Domain Analysis

The domain is the deployment configuration of a small production estate: one Kubernetes cluster of seven nodes, ten namespaces, and roughly thirty long-running processes belonging to about twenty applications. The estate is real and its current configuration is hand-maintained, which is what makes it usable as evidence: every concept below was extracted from configuration that exists, and several modelling decisions are justified by a defect the existing configuration already has.

2.1 Where the Repetition Is

Four observations from the existing configuration drive the whole language.

One fact is written down many times. The hostname of one application appears in seven authoritative places across three repositories, and two conformance tests exist for no purpose other than detecting when the seven disagree. A port appears in the container specification, in the service, in the probe and in the monitoring rule. The guard was cheaper to write than the fix, which is the situation a generated deployment set removes.

One derivation is performed many times by hand. Four deployments carry the same rollout block, arrived at independently, with the reasoning recorded in comments that no tool can read. Comments record

that a recreate strategy opened a zero-pod window, and that a JVM cold start needs a startup budget an order of magnitude larger than a static bundle's. That is a rule about a class of process, expressed four times as a constant.

Values are stated in the wrong vocabulary. The existing configuration declares mechanisms, not requirements: a rollout strategy rather than whether the application may be taken down during a cutover, a node label rather than what the process needs from a machine. A process whose author forgets to declare a recreate strategy over a volume that cannot be attached twice appears to work on a one-node cluster and fails the first time a second node exists.

Some values cannot be written by one application at all. A hostname must be unique across the estate; a graphics card is held by one process at a time; the edge is a shared, finite resource. These values are contended, and no application can decide them from inside its own repository.

2.2 The Concepts the Domain Requires

The analysis produces two groups of concepts, and the split between them is the main modelling decision of the project (Section 3).

The *structural* concepts are the ones a reader would expect: a project groups applications, an application groups the processes released together, a process offers named ports and depends on the ports of others, and an application is reachable at a hostname whose paths route to its processes.

The *operational* concepts are the ones that carry most of the generated output: what a process needs from a machine, what its data is worth if it is lost, which secrets it may read and how it receives them, what answers a health check, how long a cold start may legitimately take, and whether a new version may run beside the old one during a cutover. These have no counterpart in a structural architecture notation, and they are where the interesting derivations live.

2.3 Notation

The Task 0 feedback asked why UML class diagrams are used, and suggested ArchiMate or the C4 model as alternatives or as additions.

For the metamodels themselves the class diagram is not really a choice. A metamodel defines a language's abstract syntax, Ecore implements EMOF, and EMOF is classes, attributes, references and multiplicities, so a class diagram is the notation whose constructs correspond one-to-one to what the metamodel is built from. ArchiMate and C4 describe a system, not a language, so neither can define one.

For the *models* the suggestion is a good one, and this report takes it up. A resolved model is an arrangement of processes on machines, which is exactly what a deployment view is for, and the resolved model contains every fact such a view needs: which process runs where, which application answers on which hostname, and which process depends on which. Section 7 shows one example model that way rather than only as text, and generating an ArchiMate technology view or a C4 deployment diagram from a resolved model is a second model-to-text transformation over the same target metamodel, which Task 3 could add at little cost.

3 Modelling Decisions

Seven decisions shape both metamodels. Each is stated with the observation that forced it, because in every case the alternative had already been tried in the estate's hand-maintained configuration.

3.1 Three Model Stages, Two Metamodels

The pipeline has three stages: an authored model, a resolved model, and the generated text. Only the first two are given metamodels. The generated files are text in a target syntax that already has a schema of its own, so giving them a third metamodel would restate that schema without adding a constraint the project could check.

The three stages are deliberately *not* described as metalevels. No stage is a type model of the stage below it, so calling the arrangement a metamodel hierarchy would claim an `instanceOf` relation that does not hold. They are stages of one transformation pipeline, and the word metamodel is reserved for a language definition.

3.2 The Layer Rule

The rule that makes the middle stage worth having: **the authored model contains no mechanisms, and the generated text contains no decisions.** Every value is assignable to exactly one stage by two questions. Must a human author it? And does it record a decision, or merely serialise one?

The counter-experiment is on record. An earlier two-stage arrangement, with resolution private to the generator, produced three mutually incompatible documents all claiming the same version identifier, because a two-stage vocabulary could not say which of the three was the authoring shape. The middle model exists so that a decision has one place to live and one name.

3.3 Contention Decides Authority

A value is decided by the platform if and only if it must be unique across the estate or draws on a shared finite resource; every other value is declared by the application and carried through untouched. One question replaces a per-field negotiation.

The rule required one refinement that is worth reporting, because the naive reading is wrong. Contention decides who *arbitrates*, not who *authors*. Memory and CPU are draws on a finite pool, so a who-authors reading would forbid an application from stating them, which leaves every process with no reservation at all. The application therefore states its requirement and the platform arbitrates it against what the machines publish. The accepted cost is named rather than hidden: nothing today stops an author overstating the requirement, because the arbitration is an eligibility test against one machine rather than a sum across the estate.

3.4 Derivation Is Total, and There Is No Override

Every value the platform decides is a function of the authored model and the platform's own facts. A derived value has exactly one declaring site, so the metamodel carries no override mechanism and no exception vocabulary.

This decision was taken against evidence that initially argued for a hatch: one process carried a hand-set deadline three times smaller than its neighbours, justified by the observation that it starts in seconds rather than minutes. That is not a value only its owner could know; it is evidence that one rule over the startup budget was wrong for a whole class of process. The correct response is a rule that reads an input the process already declares. Keeping the hatch would have hidden a wrong derivation behind a per-process reason, where removing it makes the same error visible as a wrong result for the entire class.

3.5 Values Name Model Concepts, Never Target Fields

No attribute of the source metamodel, and no attribute of the target metamodel, is named after a field of the generated syntax. A process declares whether it may be taken down during a cutover, not a rollout strategy; it declares the paths it must write, not a security context. The target metamodel records the platform's decisions in the same vocabulary, and the generators are the only place a target field name is spelled.

This is what keeps the target metamodel a model rather than an object tree with extra steps, and it is checked directly: no key of either metamodel may be a field name of the target syntax.

3.6 Closed Vocabularies

Wherever an attribute admits a fixed set of values, the set is closed in the metamodel rather than typed as a string. The estate supplies the argument: one existing field was typed as a free-form string, every application wrote the same value into it, and not one of those values was correct. A field that is wrong in every observed case at no cost is a comment, not a declaration, and closing the vocabulary is what turns it back into one.

3.7 Structure Carries Atomicity; the Graph Carries Order

Two groupings are needed, and deriving one from the other was tried and rejected. What must be released together is *structural*: the processes of one application switch together, or none of them switches. What must be applied before what is *derived* from the dependency graph. Deriving atomicity from the graph instead takes the transitive closure of every edge and turns the whole estate into one unit, which would make every change estate-wide.

4 Alternatives Considered

Table 2 lists the alternatives encountered while building the metamodels, with the reason each was rejected. The four discussed in prose below are the ones where the rejection is not obvious.

Alternative	Why it was rejected
Template the target files directly (Helm, Kustomize overlays)	No abstract model, so no constraint can be written and no fact can be derived once; the repetition the project exists to remove stays.
Annotate the target files with extra fields	Tried in the estate: the authoring shape and the resolved shape end up sharing one name, and no vocabulary can say which document is wrong.
Two stages, resolution private to the generator	The generator then takes decisions that appear in no model and can be reviewed nowhere.
A third metamodel for the generated files	The target syntax already has a schema; a third metamodel restates it.
A grammar first, with the metamodel inferred	Puts the concrete syntax in charge of the concepts; the same language has to be readable from two syntaxes.
Free-form strings instead of closed vocabularies	Tried in the estate: the one free-form field was wrong in every observed case.
An override mechanism for derived values	Hides a wrong derivation behind a per-process reason instead of exposing it for the whole class.

Table 2: Alternatives Encountered and the Reason Each Was Rejected

4.1 A Third Metamodel for the Generated Files

The Task 0 proposal carried three metamodels, one per stage of the pipeline. It was reduced to two, and this is the clearest scope reduction the feedback produced.

The argument for a third metamodel is that it would let the generation step be checked structurally rather than as text. The argument against it is that the target syntax is already schema-defined, so the third metamodel would be a hand-maintained restatement of a schema that exists, with a drift problem of its own and no constraint that the second metamodel cannot already express. The generated files are therefore compared as text against reviewed expected output, and the typed target resources live inside the second metamodel, where the generation templates can walk them.

4.2 Grammar First or Metamodel First

The source language has a textual concrete syntax, so the language could have been defined by a grammar with the metamodel inferred from it. It was defined the other way round: the metamodel first, with the grammar as one syntax over it.

The reason is that the same source model must be readable from two concrete syntaxes, and the production implementation parses a different serialisation of the same language. If the grammar were in charge, the two would agree only by coincidence. With the metamodel in charge, both syntaxes are checked against one structure, and that structure is exported as a machine-readable description that both implementations are held to.

4.3 One Resolved Model, or One Per Application

The resolved model covers the whole estate in one document, rather than being produced per application. Some of what it records cannot be computed for one application alone: which shared edge carries a hostname, the order in which units are applied, and what an application derives from the applications that depend on *it*. Producing a per-application model directly would either recompute those properties inconsistently or omit them.

What an application’s own repository receives is therefore a *projection*: the slice belonging to that application, obtained by filtering the estate-wide model rather than by a separate computation, so the two cannot disagree about what was decided.

5 The Source Metamodel

The source language is authored in two documents that together form one metamodel. A *project* document is written by the owner of a group of applications and states what those applications need. A single *platform* document is written once for the estate and states what the estate offers. The split follows directly from the contention rule of Section 3: a value that must be unique across the estate, or that draws on a shared finite resource, is stated in the platform document, and everything else is stated by the application that needs it.

5.1 The Project Document

Figure 1 shows the project half of the source metamodel. Its root is a **Project**, which carries an owner and the applications it groups. An **Application** is the unit that is released as a whole: it carries an identifier that is unique across the estate, the processes released together, the hostnames it is reachable at, the secrets it holds, and, optionally, how its failures should be alerted.

A **Process** is the level at which almost every operational concern sits. It names the image it runs, what kind of thing it is, which named ports it offers, what it needs from a machine, which paths it must write, what answers a health check, what its volumes are worth, and whether a new version may run beside the old one during a cutover. The concerns that belong to the application rather than to one of its processes are kept on the application: a hostname fronts several processes and picks between them by path, so exposure cannot sit on a process without being written twice.

A **Process** carries almost every operational concern, so it is shown in four figures rather than one: Figure 2 its capacity, helpers and health, Figure 3 what it needs from a machine, Figure 4 its storage and files, and Figure 5 its configuration, dependencies and the ports it offers. All four are cut from the same source as Figure 1.

Table 3 lists the classes and what each one is for.

Class	What it represents
Project	The authored unit: an owner and the applications the document groups.
Application	What is released as a whole, identified uniquely across the estate.
Process	One containerised process: image, kind, ports, needs and health.
Surface	A named port a process offers, declared once.
Exposure	A hostname the application is reachable at, and who may reach it.
Route	A path on that hostname, and the process and surface that serve it.
Placement	What a process needs from a machine, as hard requirements.
Volume	Storage a process holds, with what its data is worth if lost.
Probe	What answers readiness or liveness for a process.
Dependency	An edge to another application's named surface.
Grant	A secret a process may read, how it is delivered, and its rotation.
Asset	A file a process needs that is configuration rather than code.
Observability	Where metrics are collected and how urgently failures alert.
Sidcar	A helper process sharing the main process's lifetime.
Capacity	A count above one, with the reason it is not derived.

Table 3: Classes of the Project Document

5.2 The Platform Document

The platform document carries the facts and policies that no single application may decide: what the cluster is, where the edge terminates and which audiences each edge tier serves, one backup policy per durability class, one backup method per engine, one hardening posture, one probe cadence, one monitoring cadence, and the services the estate runs without deploying them.

5.3 Closed Vocabularies

Wherever an attribute admits a fixed set of values, the metamodel declares an enumeration. The project document contributes vocabularies for the kind of a process, its runtime, its engine, its cutover requirement, a volume's durability class, machine architecture and storage medium, alert urgency, the audience of an exposure, a content policy, a path match, a secret's access tier and delivery, and a rotation tolerance. The platform document contributes vocabularies for the cluster's datastore and policy controller, an edge tier's listener and certificate source, and the hardening class.

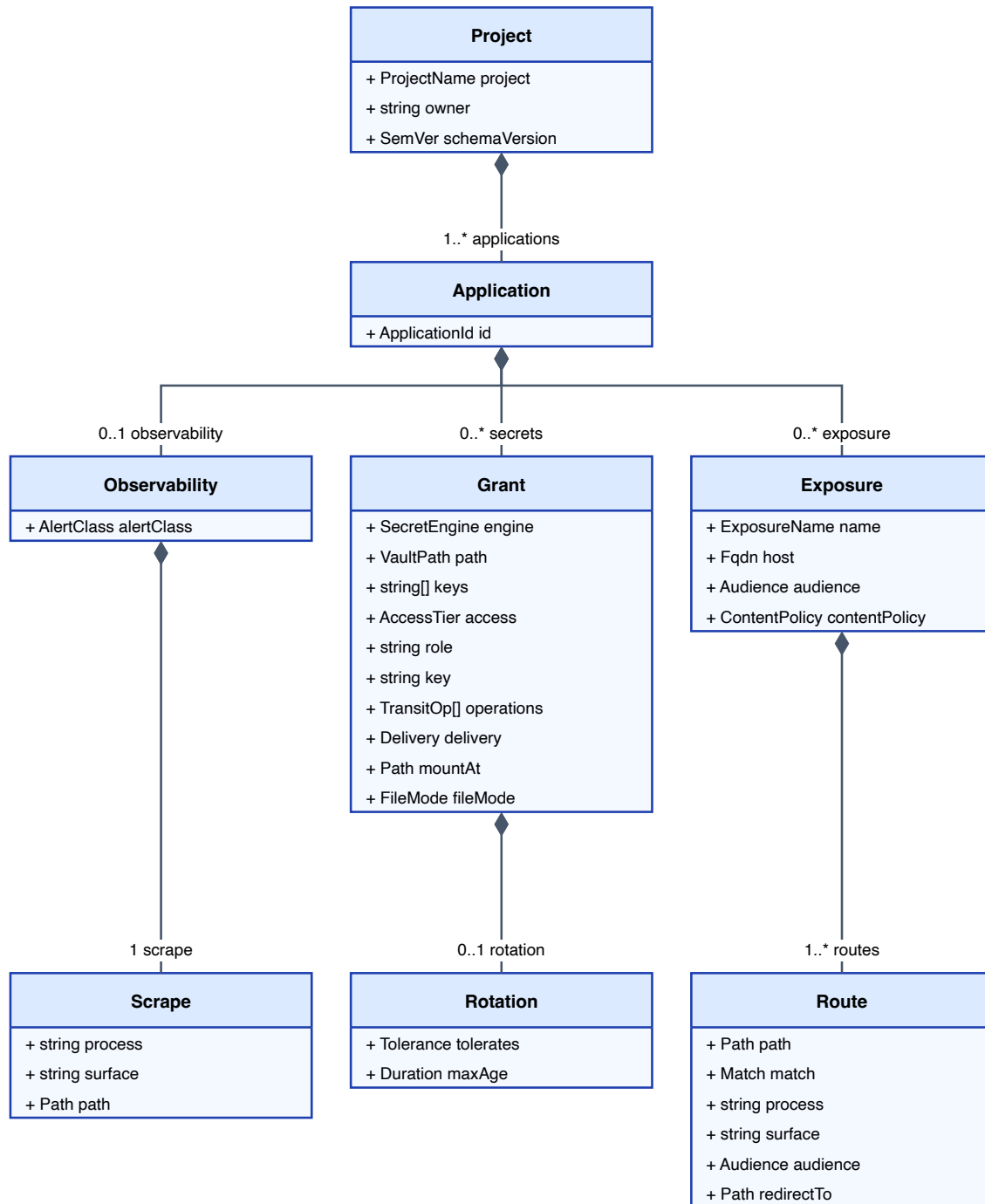


Figure 1: The Project Document: the Application and What Belongs to It

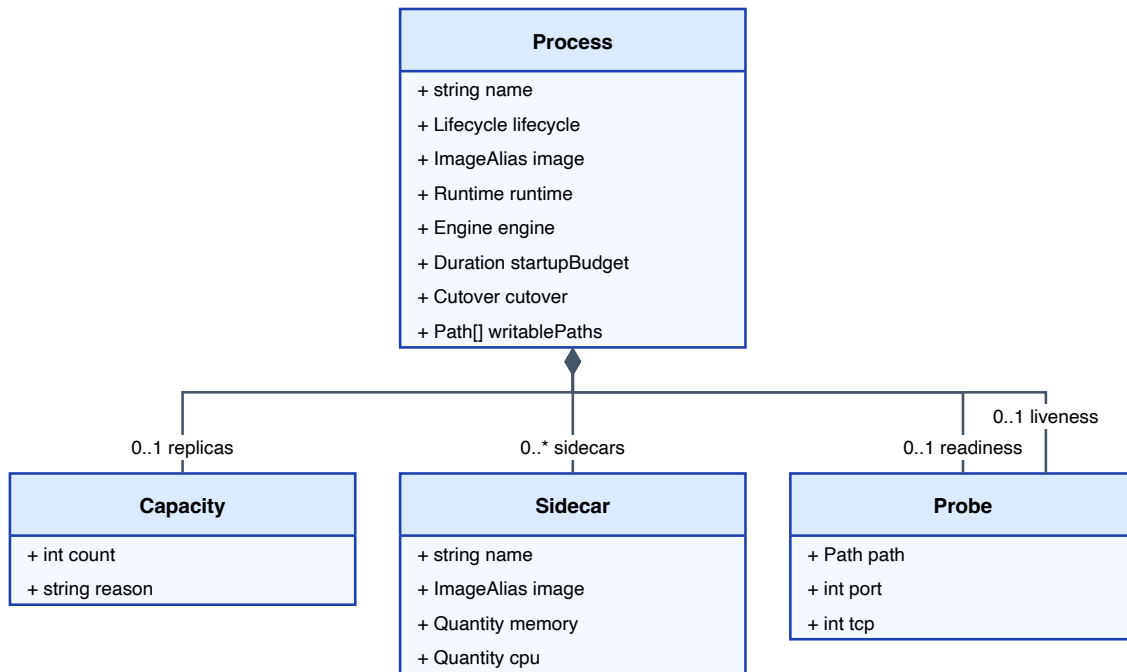


Figure 2: A Process: Capacity, Helpers and Health

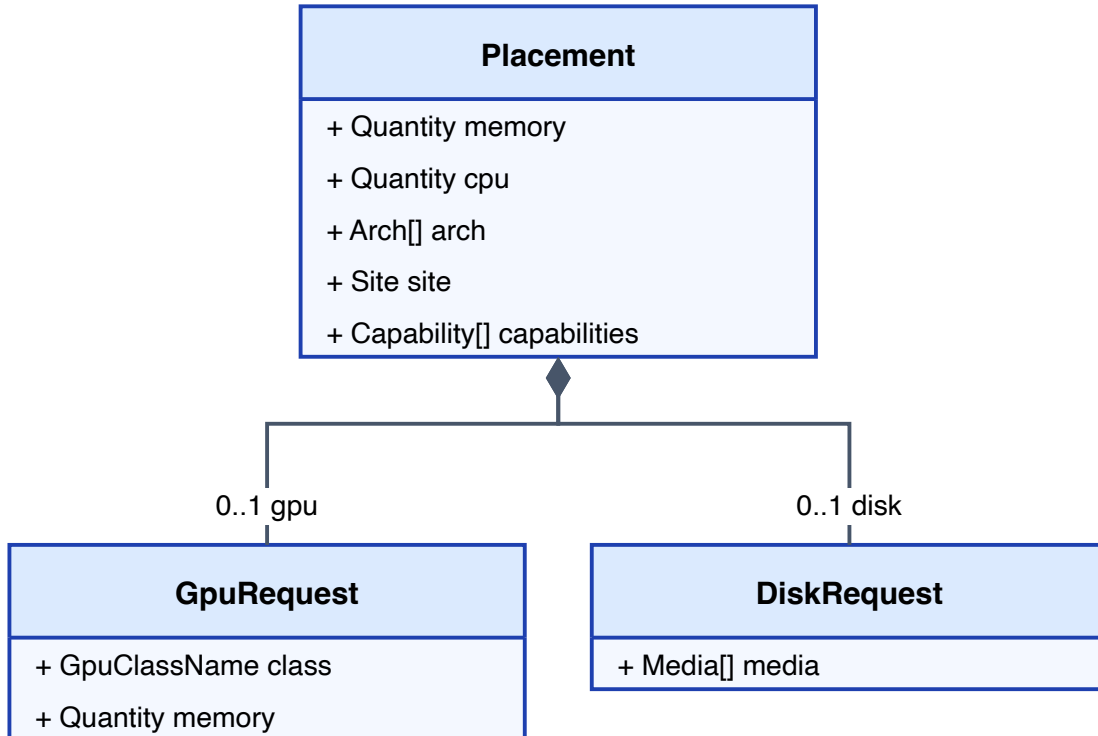


Figure 3: A Process: What It Needs From a Machine

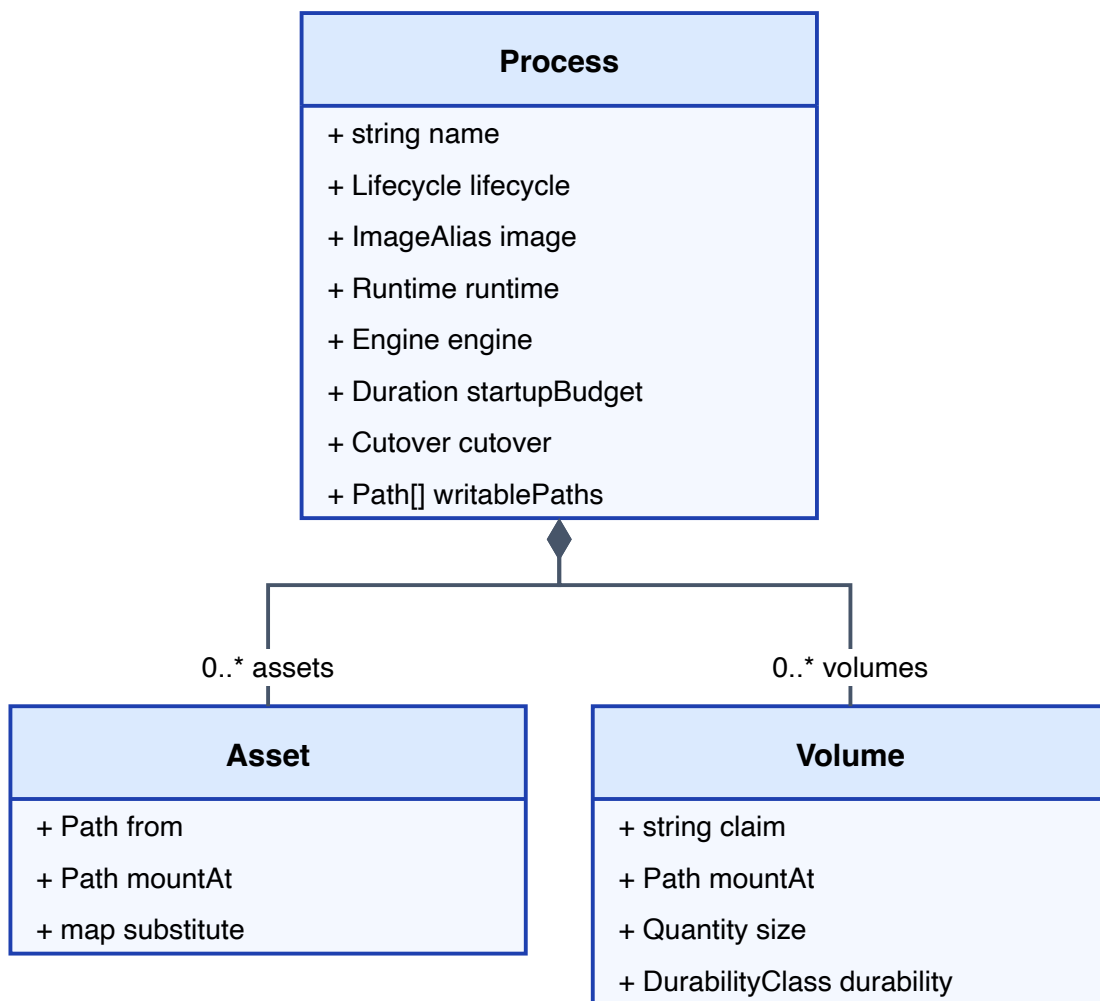


Figure 4: A Process: Storage and Files

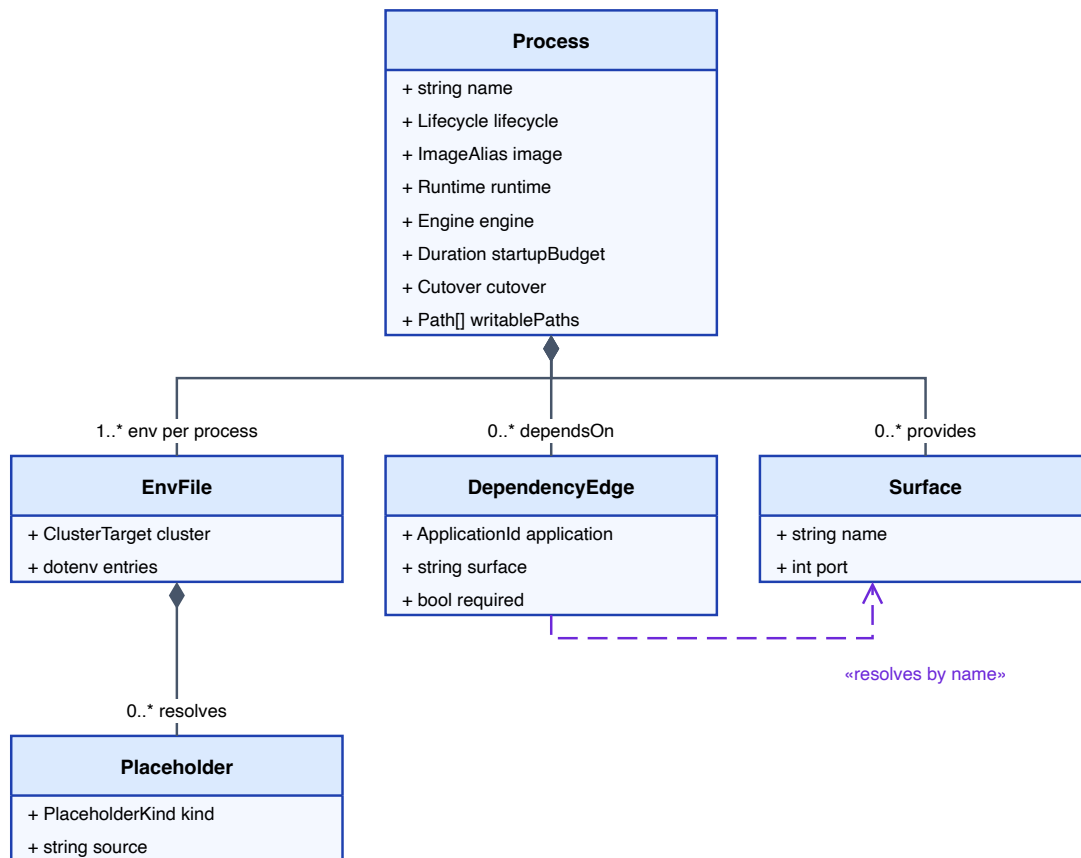


Figure 5: A Process: Configuration, Dependencies and the Ports It Offers

The enumerations are the surface most of the constraints are written against, which is the argument of Section 4 against free-form properties: a constraint can enumerate an enumeration exhaustively, and cannot enumerate a string.

6 The Target Metamodel

The target metamodel is the language the model-to-model transformation of Task 2 writes into, and the language the code generation of Task 3 reads. It holds every decision the platform took, recorded in the same vocabulary as the source metamodel, together with the typed target resources and the output path of each one.

6.1 What It Must Record

A resolved model is complete in a specific sense: the generators may read it and nothing else. Anything a generator would otherwise have to decide for itself is recorded here, because a decision taken while serialising appears in no model, is reviewed nowhere, and is invisible in the projection an application's owner reads back.

Three groups of content follow from that.

Decisions. Which namespace an application lives in; which edge tier carries each hostname and what the middleware chain in front of it is; the precedence of each route; the eligible machines for each process and the machine a bound volume already ties it to; the identity of each process and the secret paths it may read; the resolved image digest; the derived probe timings, the deadline, the instance count and the capacity; the backup plan each volume's durability class earns; and the resolved dependency edges with the policy peers they permit.

Order and atomicity. The units in which the estate is applied, with the edges that say which precedes which, and, per application, the inputs of the gate that decides when a new version may receive traffic.

Provenance. The hash identifying the render, and one digest per pinned input: every project document, the platform document, the machine contract, the image lock and the observed-state snapshot. This is what makes reproducibility a checkable claim rather than an assertion: identical recorded digests must produce identical output, so a difference under identical digests is a defect in the generator.

6.2 Structure

The metamodel is shown in five figures, each cut from one source by the same script that cuts the source metamodel's figures.

The root is a **ResolvedDeployment** covering the whole estate, because several of its properties are estate-wide and cannot be computed for one application alone. It carries the provenance, the resolved applications, the units and their ordering, and the path plan.

A **ResolvedApplication** is both the element the estate-wide model contains and the document published back to an application's own repository. In the second role it carries its own provenance; nested in the estate-wide model it does not, because the enclosing document's provenance already covers it. This is what makes the projection a filtering of the estate-wide model rather than a second computation, so the two cannot disagree.

Below it, a **ResolvedProcess** carries the per-process decisions, a **ResolvedExposure** carries the hostname with its tier, middleware chain and ordered routes, and the volumes, grants, dependency edges and writable paths each carry what the platform decided about them.

6.3 The Path Plan

Every output path is assigned here, not by the generator that writes the file. The model carries, for each object to be generated, the generator that owns it and the path it is written to.

The rule follows from the layer rule rather than adding to it, and two observed cases show that the alternative does not work. An object that belongs to a group of applications rather than to one of them is emitted once per application when the path is computed per generator, producing several identical objects at several paths. And an estate-wide object lands in whichever namespace the generator that emitted it happened to key off. Assigning the paths in the model also moves the check that two generators never claim one path to a point where the complete set of paths is known, which is before any generator runs.

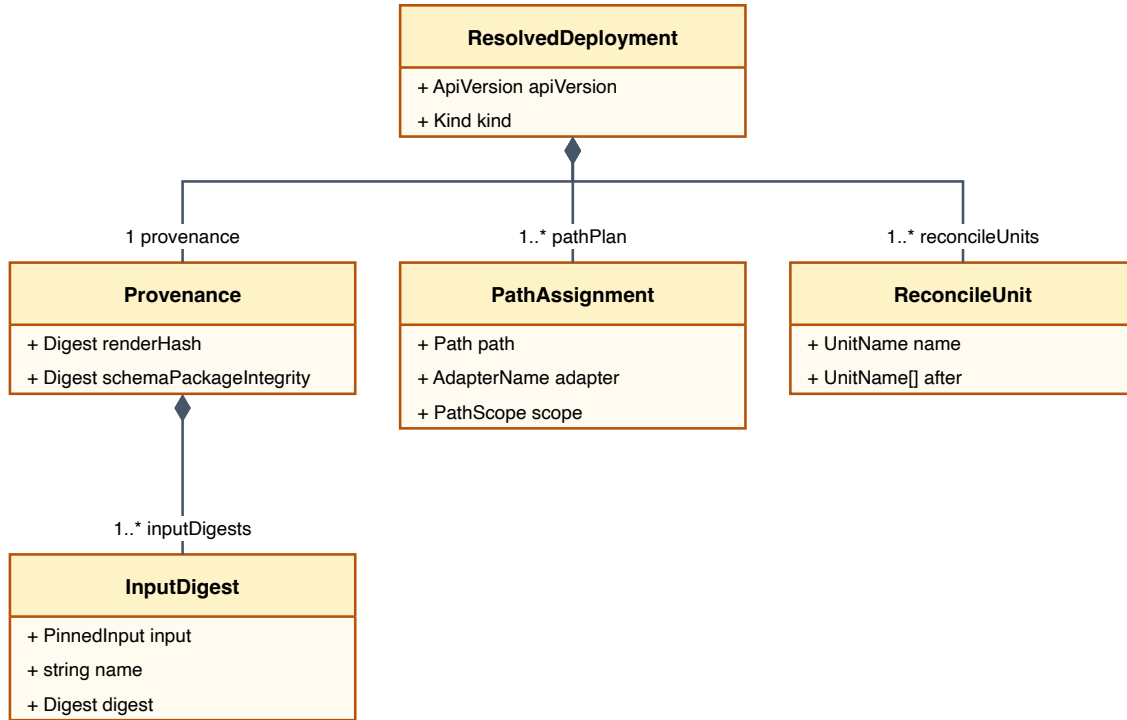


Figure 6: The Estate-Wide Frame: Provenance, the Path Plan and the Units

6.4 The Generated Resources

The generation step has a closed set of six generators, one per subsystem: the workloads and their supporting objects, the network policy, the monitoring objects, the edge routing, the secret-store policies, and the secret delivery objects. The set being closed is the scope commitment of Section 1: adding a seventh is a recorded decision, not a new feature.

6.5 No Target Field Names

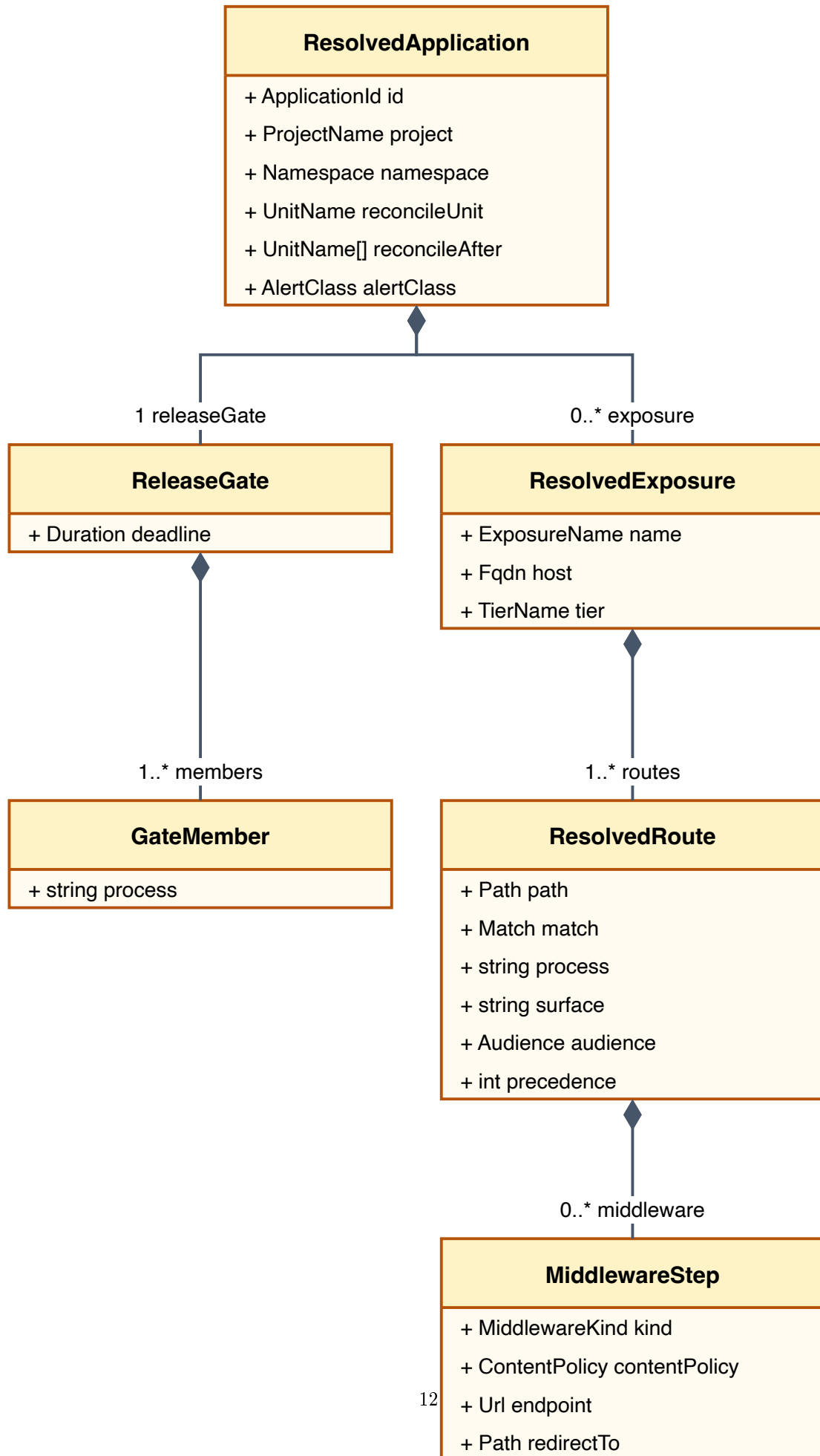
The constraint of Section 3 applies most visibly here. The resolved model records that a process may not be taken down during a cutover; it does not record a rollout strategy. It records the hardening posture and the paths the process must write; it does not record a security context. It records the middleware chain as a sequence of named steps; it does not record the target proxy’s spelling of them. Each generator is the one place its own target vocabulary appears, and that separation is checked directly rather than left to review.

7 Example Models

Four example models instantiate the source metamodel, and each is chosen to exercise something the metamodel claims. They are committed beside the metamodels, together with the expected result of parsing each one, so that the examples are checked rather than illustrative.

7.1 The Minimal Model

The minimal model is the one to read first. It declares a project with one application and one process, and it is shorter than the file the same deployment needs today by roughly an order of magnitude. What it does *not* declare is the point: no rollout strategy, no resource limits, no security context, no labels, no selector, no namespace, no probe timings, no instance count. Each of those either derives from something the model does declare, or is platform policy, or is refused outright.



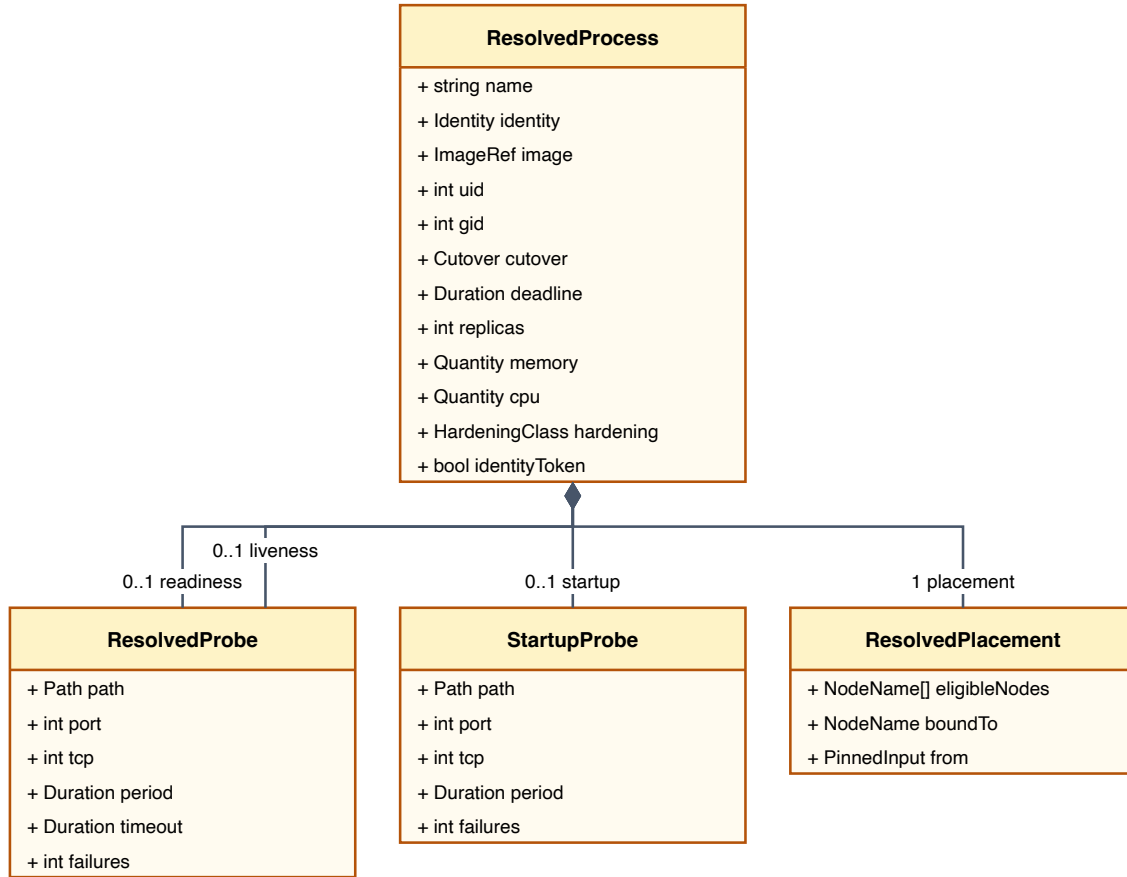


Figure 8: A Resolved Process: Health and Placement

Example	What it exercises
minimal	The smallest complete model: one project, one application, one process, and nothing that is not required. Everything absent from it is either derived or refused, which makes it the model that shows what the language forces an author to say.
auth	Two processes in one application, so the release gate has more than one member and the gate's deadline is a maximum over members rather than a single value.
data	Several applications in one project sharing a namespace, with storage and the backup plans their durability classes earn.
knowledge	Dependencies across applications, secrets, a volume already bound to a machine, and an exposure whose routes carry different audiences.

Table 4: The Example Models and What Each One Exercises

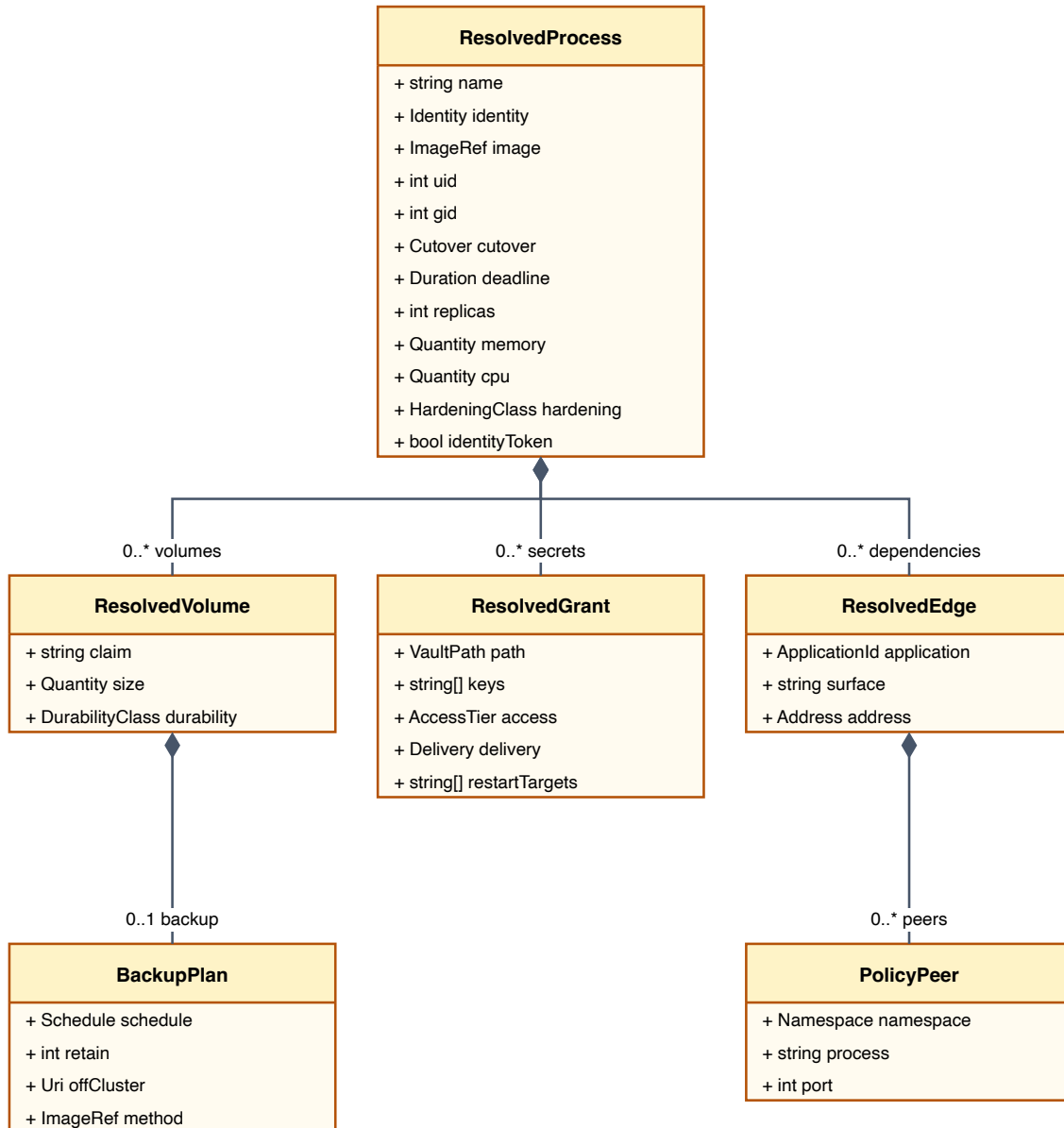


Figure 9: A Resolved Process: Storage, Secrets and Dependencies

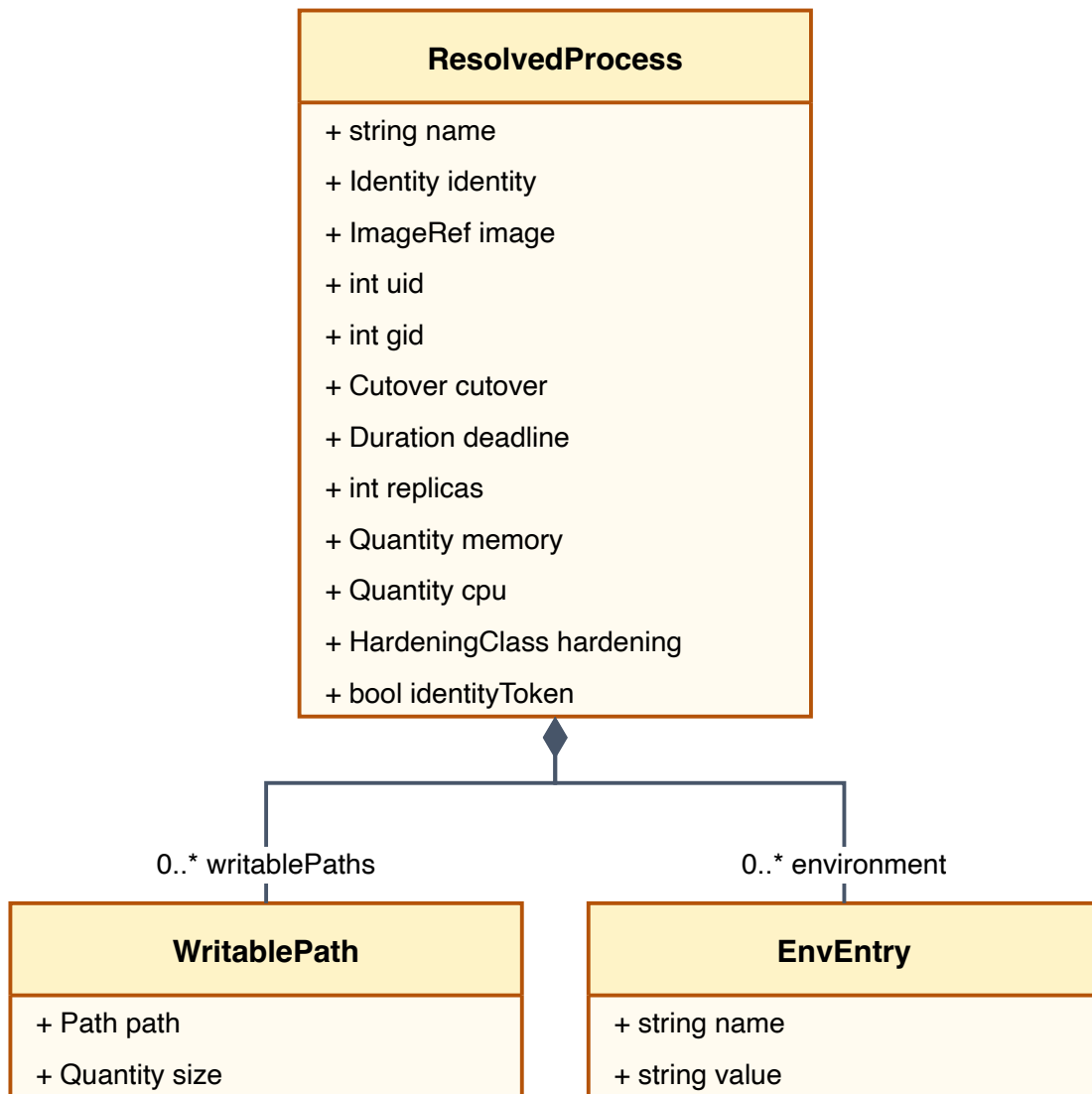


Figure 10: A Resolved Process: Writable Paths and Configuration

7.2 Refused Models

Alongside the valid examples, a set of deliberately invalid models is committed, one per constraint, each with the diagnostic it must produce. A constraint that has only ever been satisfied is not evidence that it fires, so each constraint is paired with a model that violates it.

The refused set covers, among others: a route naming a surface its process does not offer; two routes on one hostname claiming the same path; an alert class with nothing to collect metrics from; a process declaring a cutover its storage cannot honour; a secret delivery that the cluster's encryption setting forbids; a durability class with no engine to back it up; and an exposure whose audience no edge tier serves.

7.3 A Resolved Model as a Deployment View

Figure 11 shows the `knowledge` example's resolved model the way a deployment diagram would draw it: two processes placed on two machines, the volume one of them is bound to, the edge tier and hostname in front, and the two platform services they depend on. Nothing in the picture is new information. Every box and every arrow is read straight out of the resolved model, which is the point: the model is complete enough that a deployment view is a rendering of it rather than a separate document somebody keeps in step by hand.

This is the part of the Task 0 feedback that is worth acting on. A structural notation is the right one for an arrangement, and generating this view from the target metamodel is another model-to-text transformation, no different in kind from the one that generates the deployment files.

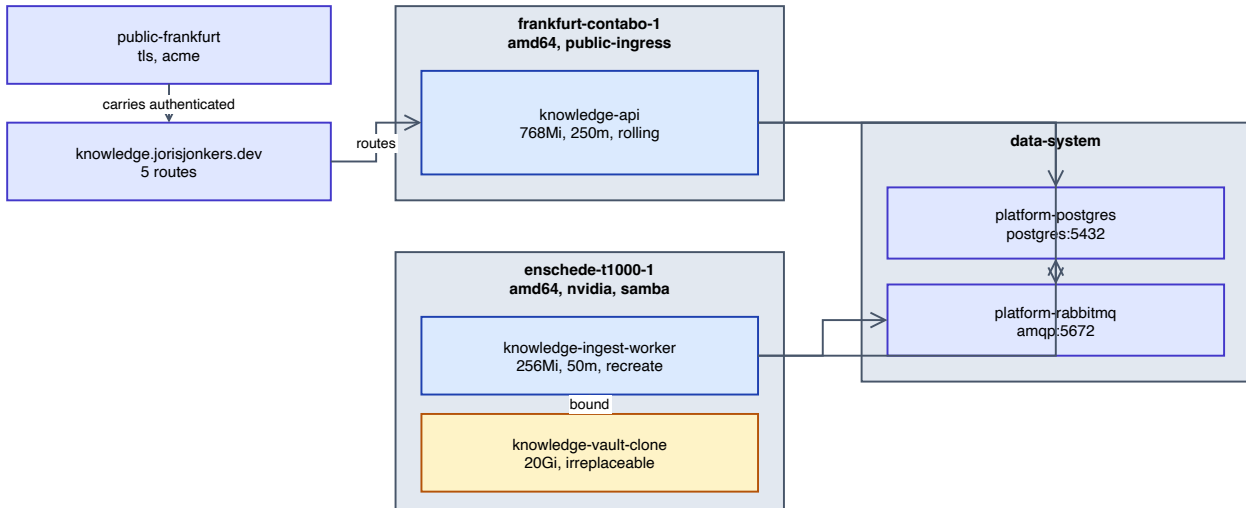


Figure 11: The knowledge Example's Resolved Model, as a Deployment View

7.4 Example Target Models

A hand-written target model accompanies the minimal example: the resolved model that the Task 2 transformation must produce from it, written and reviewed by hand rather than generated. It is the expected output of the transformation and the input the Task 3 templates are first run against, which is what lets both tasks be checked before either is complete.